Development Plan
EyeCanDo

Team 2, Hindsight
Mohammad Bilal
Ethan McMehen
Jackson Beach
Jake Finlay
Ahsan Muzammil

Table 1: Revision History

Date	Developer(s)	Change
2026/09/25	Mohammad Bilal	Completed sections 1-4, Team Charter, & reflection
2026/09/25	Jake Finlay	Completed Section 10: Proof of Concept Demonstration Plan and Reflection
2026/09/26	Jackson Beach	Completed sections 9, 11, 12, and individual reflection
2026/09/27	Mohammad Bilal	Updated license link
2026/09/27	Ahsan Muzammil	Completed initial draft of Workflow plan, Coding Standards, and Reflection (Ahsan)
2026/09/27	Ethan McMehen	Completed Sections 5, 6, 7, Reflection
2026/09/30	Jake Finlay	Reduced PoC scope and added mitigation strategies for an unsuccessful PoC (peer review feedback)
2026/10/14	Jake Finlay	Updated PoC plan for the new game-based requirements (sections, levels, and quizzes)
2026/10/25	Ethan McMehen	Updated the team member roles to have distinguished leaders

1 Introduction

This document outlines the team's development plan for the project. Sections 2–4 cover confidentiality, intellectual property, and copyright, while Sections 5–10 describe team processes, workflow, scheduling, and the proof of concept. Sections 11–14 cover the expected technologies, use of AI and LLMs, and coding standards. The Team Charter outlines the team's expectations for attendance, accountability, teamwork, and decision making.

2 Confidential Information?

There is no confidential industry information involved in this project. However, the team may interact with patients, parents, clinicians, and other stakeholders during the development and evaluation of the application. Any personal, medical, or identifying information shared during these interactions will be treated as confidential and will not be included in the project repository or documentation in an identifiable form. The team will follow the privacy and confidentiality requirements provided by the project supervisors and McMaster University when collecting, handling, or discussing stakeholder information.

3 IP to Protect

The intellectual property created for this project will solely consist of the source code, documentation, and application content developed by the team. These materials will be protected under the copyright license described in Section 4. No additional intellectual property protection, such as a trademark agreement, is currently anticipated for the project.

4 Copyright License

The copyright license adopted by the team for this project is the MIT License. This license allows others to use, copy, modify, merge, publish, distribute, sublicense, and sell copies of the software, provided that the original copyright notice and permission notice are included. The copyright license for the project can be found in the team repository.

5 Team Meeting Plan

Weekly meetings will be held in person from 3:30 PM - 4:30 PM on Wednesdays, which is conveniently during the Capstone tutorial time. These weekly meetings will proceed in-person on campus. Should a team member not be able to attend the in-person meeting they should provide at least 1 hour notice and join online (unless an emergency ensues). Each member is required to be prepared for the meeting by reading through the associated deliverable documentation beforehand. Additional virtual meetings may be required when we approach a deliverable deadline and the team needs to convene. Supervisor meetings will be held weekly, biweekly, or as needed basis which is also dependent on the availability of the supervisor. All members can chair and record their meeting minutes, but the primary recording responsibility will fall to the Team Liaison. The chair should use the agenda and meeting minutes template below. The template will be used and made into a GitHub meeting issue that can be edited by any member of the team.

Meeting Agenda

- Date, Time
- Attendance
- Purpose of Meeting Description
- New Deliverable Tasks:

  Brief introduction

Decide division of labour

Record task assignment, record miscalleneous task assignment

- Current Deliverable:

Discussion of progress

Team Member Updates

Discuss inquiries for supervisor

Discuss inquiries for professor

Meeting Minutes

- Date, Time
- Introduction
- Suggested Improvements
- Positives + Negatives discussion
- Division of Labour
- Task Assignment Record
- Discussion of Progress
- Team Member Updates
- Questions for Supervisor/Professor (Communicated by Team Liaison)
- Mental health check, satisfaction levels: Awesome, Good, Neutral, Bad, Despaired
- Extra Comments

6 Team Communication Plan

Our team communication plan will involve the following technologies and plan:

- Issues: GitHub/GitHub Projects
- Weekly Meetings: In-Person on Campus (Discord Channel if Online with ¿1 Hour Notice)
- Meeting Minutes Outside Weekly Meetings: Discord
- Meeting Agenda and Minutes: GitHub Meeting Issue
- Meeting with Stakeholders: In Person or Zoom (Dependent on Availability of Stakeholder)
- General Discussion: In-person or Discord

To avoid last minute communication or lost communication, at least one team member must respond to a message within 24 hours. If urgent communication shall arise about a deliverable (or actual emergency) than direct messaging can be performed.

7 Team Member Roles

See Table 2 for the role assignments of our team members.

Editor: This member will read through all deliverables and be responsible for the final read-through before submission. They are responsible for ensuring consistency, clarity in the documentation and conciseness in the writing within the document and can edit any text to ensure these communication goals are met.

Team Liaison: This member is responsible for any and all communication with supervisors and professors, communicating the information back to the team. This member is also primarily responsible for the meeting agenda and meeting minutes.

Technical Manager: They will be responsible for task workload assignments and for enforcing the GitHub workflow plan is followed to the standard agreed upon by the team.

Researcher: A researcher is responsible for performing initial or preporatory research and communicating the findings back to the team, and if required into the CD documentation.

Back-end Developer: They are responsible to ensure the back-end is functional and meeting the coding standards agreed upon by the team.

Front-end Developer: They are responsible to ensure the front-end is functional and meeting the user-centric interface standards, needs, and goals of our users.

Comment Reviewer: This member is responsible for reviewing the clarity and presence of effective code comments.

GitHub Wiki Manager: They are responsible to verify the GitHub Wiki Manager contains all the expected content for presentation.

Table 2: Team Member Roles & Assignments

Role	**Name**	**Change Possibility (Y/N)**
Editor	All	Y
Team Liaison	Ethan	N
Technical Manager	Ahsan	Y
Lead Researcher	Jackson	Y
Researcher	All	N
Lead Back-end Developer	Bilal	Y
Back-end Developer	All	N
Lead Front-end Developer	Jake	Y
Front-end Developer	All	N
Code Reviewer	All	N
GitHub Wiki Manager	All	Y

8 Workflow Plan

Our development workflow is designed to support parallel work while maintaining traceability, code quality, and controlled integration of changes. GitHub will be used for version control and project management, with GitHub Issues, Projects, Pull Requests, and Actions working together throughout the development process.

8.1 Git Workflow

GitHub will be used as our primary version control system for managing source code, LaTeX documentation, and other project artifacts. Our Github repository link can be found here: GitHub Repository Link

8.1.1 Repository Organization

We created a GitHub organization for the project to provide centralized management of the project's repositories, permissions, and team access. Using an organization prevents the project from being dependent on an individual team member's personal repository and provides a consistent location for all project resources.

Our repository structure follows a fork based workflow:

1. The team maintains a primary repository containing the project's main codebase and documentation.

2. Each team member maintains a fork of the primary repository.
3. Team members create task specific branches within their fork.
4. Completed work is submitted to the primary repository through Pull Requests (PRs).

We chose this approach to allow team members to work independently while protecting the primary repository from direct changes. It also provides a formal review process before changes are integrated into shared branches.

8.1.2 Branch and Pull Request Workflow

Each task will be completed on a dedicated branch. This workflow applies to different types of work, including:

- New features
- Bug fixes
- Enhancements
- Documentation and reports
- Testing
- Other project tasks

Team members will make commits throughout their work so that changes remain traceable to the corresponding task or issue.

The overall branch and Pull Request workflow is shown in Figure 1.

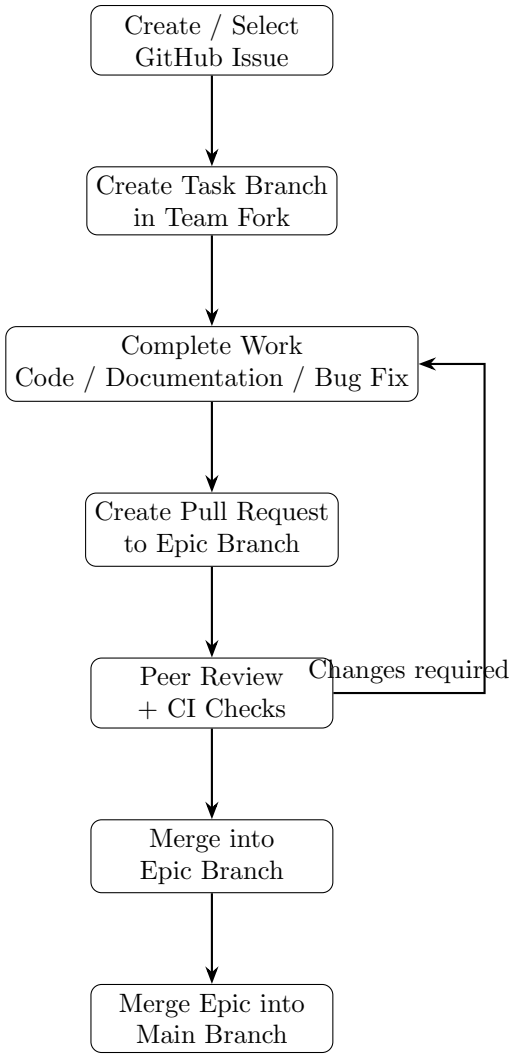


Figure 1: Git and Pull Request Workflow

Once work is complete, the branch will be submitted as a PR to the appropriate epic branch in the primary repository. The PR will reference the GitHub Issues associated with the work. This provides traceability between the task being completed and the changes submitted to the repository.

PRs will require peer review before being merged. Reviewers will verify that:

- The changes satisfy the associated issue.
- The changes follow the team's development standards.
- The implementation or documentation does not introduce unintended problems.
- Required CI checks have passed.

After approval, the PR will be merged into the appropriate epic branch. Epic branches allow related work to be integrated and reviewed together. Once an epic has been completed, its branch will be merged into the main branch.

This workflow provides multiple quality control points while still allowing team members to work independently.

8.1.3 Commit Message Convention

To maintain consistent and traceable commit history, the team will follow a standard commit message structure:

```
<type>(<scope>):  <message> (#<issueNumber>)
```

The *Type* identifies the general purpose of the change, while the optional *Scope* identifies the specific part of the project affected. The *Message* briefly describes the change, and the issue number provides traceability between the commit and the corresponding GitHub Issue.

The following commit types will be used:

Type	**Purpose**
feat	Addition of new functionality.
bug	Correction of unexpected or incorrect behaviour.
imp	Improvement to existing functionality.
docs	Changes to documentation or project deliverables.
test	Addition or modification of tests.
research	Research or investigation related to the project.
refactor	Restructuring of existing code without changing its intended behaviour.
perf	Improve performance to existing code.

Table 3: Git Commit Types

For documentation related commits, the optional scope will identify the document being modified. For example, `DP` may be used for the Development Plan and `PS` may be used for the Problem Statement.

Examples of valid commit messages include:

Listing 1: Example Commit Messages

```
feat: add eye drop preparation activity (#42)
bug: fix activity completion state (#57)
docs(dp): update workflow plan (#31)
docs(ps): revise problem statement (#18)
test: add activity completion tests (#63)
research(ar): investigate AR procedure simulation (#71)
```

This convention provides a consistent commit history and makes it easier to identify the purpose of a change and trace it back to the corresponding GitHub Issue. Pull Requests will additionally reference or close the associated issues where appropriate.

8.2 Issue Management

GitHub Issues will be used as the primary mechanism for tracking project work. Issues will be used for development tasks, bugs, documentation, research, meetings, reviews, and other activities.

Each issue will contain, where applicable:

- A clear description of the task.
- The expected outcome or acceptance criteria.
- An assigned team member.
- Relevant labels.
- A milestone.

- Links to related issues or Pull Requests.

8.2.1 Breaking Down Work

Large tasks will be divided into smaller issues or sub issues using a divide and conquer approach (as mentioned in section 9). For example, a large documentation deliverable may be divided into individual sections that can be assigned to different team members.

This allows the team to work in parallel while keeping the relationship between smaller tasks and larger deliverables visible.

8.2.2 Kanban Board

We will use a GitHub Projects Kanban board to track the current state of issues throughout the workflow. The stages used by the team are shown in Table 4.

Stage	**Purpose**
Backlog	Work that has been identified but is not yet scheduled.
To Do	Work that is ready to be started.
In Progress	Work currently being completed by a team member.
In Review	Work that has been completed and is awaiting review and/or CI checks.
Done	Work that has been reviewed, approved, and completed.

Table 4: GitHub Projects Kanban Workflow

The Kanban board provides the team with a shared view of project progress and helps identify outstanding work and potential bottlenecks.

8.2.3 Milestones

Milestones will be used to group issues according to major project deadlines, deliverables, or development phases. This allows us to track progress toward larger objectives rather than only individual tasks.

For example, a milestone may contain all issues required to complete a specific project deliverable or prototype. This provides a high level view of whether the team is on track to complete upcoming objectives.

8.2.4 Issue Templates

We will use GitHub Issue Templates for recurring activities. Templates provide a consistent structure and reduce repetitive work when creating issues.

The templates used by the team are shown in Table 5.

Template	Purpose
Lecture	Record important information, notes, attendance, and action items.
Peer Review	Record feedback and follow up actions from peer reviews.
Supervisor Meeting	Record agendas, discussions, decisions, and action items from supervisor or stakeholder meetings.
TA Meeting	Record agendas, feedback, decisions, and action items from TA meetings.
Team Meeting	Record discussions, decisions, assigned tasks, and follow up actions from team meetings.
Blank Issue	General purpose template for tasks that do not fit another template.

Table 5: GitHub Issue Templates

8.2.5 Issue Labels

Labels will be used to classify issues according to the type of work they represent. Labels describe the nature of the work rather than its current status, since status is managed through the Kanban board.

The labels used by the team are shown in Table 6.

Label	Purpose
Bug	Unexpected or incorrect behaviour requiring correction.
Feature	New functionality or a significant addition to the system.
Enhancement	Improvement to existing functionality.
Documentation/Deliverable	Reports, documentation, presentations, or other deliverables.
Testing	Creation, modification, or execution of tests.
Research	Research or investigation required to support project decisions.
Lecture	Lecture related tasks and notes.
Peer Review	Peer review activities and resulting actions.
TA Meeting	TA meeting notes, feedback, and actions.
Team Meeting	Team meeting discussions and actions.
Supervisor Meeting	Supervisor or stakeholder meetings and follow ups.

Table 6: GitHub Issue Labels

8.2.6 Pull Request Template

To maintain consistency across Pull Requests, we will use a standard PR template. The template will require contributors to identify the associated issues, summarize their changes, and confirm that the required checks have been completed.

An example of the template is shown below.

Listing 2: Pull Request Template

```
## Description

Briefly describe the changes made in this Pull Request.

## Related Issues
```

```
- Closes #IssueNumber
- Related to #IssueNumber

## Type of Change

- [ ] Feature
- [ ] Bug Fix
- [ ] Enhancement
- [ ] Documentation
- [ ] Testing
- [ ] Other

## Testing

Describe the tests or verification performed.

## Checklist

- [ ] Changes satisfy the associated issue(s).
- [ ] Code/documentation follows team standards.
- [ ] Tests have been added or updated where necessary.
- [ ] All CI checks pass.
- [ ] Changes have been reviewed locally before submission.

## Additional Notes

Include any additional information reviewers should know.
```

A completed example is shown below.

Listing 3: Example Completed Pull Request

```
## Add Eye–Drop Preparation Activity

### Related Issue

Closes #42

### Description

Added the eye–drop preparation activity and associated user interface.
The activity introduces the procedure and provides an interactive
simulation for the child.

### Type of Change

– [x] Feature
– [ ] Bug Fix
– [ ] Enhancement
– [ ] Documentation
– [ ] Testing
– [ ] Other

### Testing

Tested the activity in the Unity editor and verified that the activity
can be completed from beginning to end without errors.

### Checklist

– [x] Changes satisfy the associated issue(s).
– [x] Code/documentation follows team standards.
– [x] Tests have been added or updated where necessary.
– [x] All CI checks pass.
– [x] Changes have been reviewed locally before submission.

### Additional Notes

No additional notes.
```

8.3 CI/CD

GitHub Actions will be used for continuous integration (CI) and, where appropriate, continuous delivery (CD). The purpose of the CI pipeline is to automatically verify changes before they are merged, reducing the likelihood of introducing formatting issues, failing tests, or broken builds into shared branches.

The overall CI pipeline is shown in Figure 2.

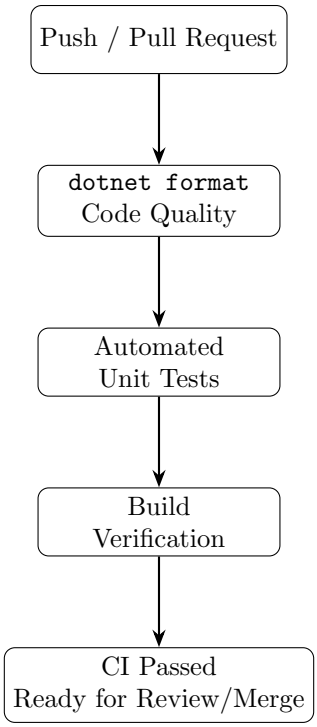


Figure 2: Continuous Integration Pipeline

The pipeline will perform the following checks:

1. **Code Quality:** `dotnet format` will verify that .NET code follows the team's formatting standards.
2. **Unit Tests:** Automated tests will be executed to verify individual components. GameCI will provide a headless Unity environment for running Unity tests.
3. **Build Verification:** Where applicable, the project will be built to ensure that changes do not prevent the application from compiling or generating a valid build.

The CI workflow will run when relevant branches are updated and when PRs are created or updated. Required CI checks must pass before a PR can be merged.

For continuous delivery, GitHub Actions may be used to automatically generate builds or project artifacts for testing, demonstrations, or milestones. This will reduce manual build steps and provide a consistent process for producing validated project versions.

9 Project Decomposition and Scheduling

How will we be using GitHub Projects?

We have created a Kanban board using GitHub Projects for our capstone. Currently, we have four columns: Backlog, In Progress, Review, and Done. We meet as a team to create issues for the upcoming deliverables, which adds them to the Backlog column, as the project is linked to our repository. During these meetings, we divide up the work and assign it in a relatively equal manner. Each team member moves the issues they are assigned to through the different stages of the project as they work on them. For sections with multiple subsections, we create a parent issue for the entire section, and sub-issues under said parent correlating to the various sub-sections. This allows multiple people to tackle a section in a straightforward manner (in terms of project scheduling). We also have alphabetic codes for the deliverables, to serve as an additional organization tactic (e.g., the Problem Statement document is "PS"). This allows for keyword search on the

Kanban board, to see what work needs to be completed for a specific document. The project will also be used to keep track of meeting minutes, as well as lecture attendance for the group.

We create milestones as well, which correspond to the deliverable deadlines as laid out in the course schedule. We currently only have the first milestone created, however there will be a milestone for each delivery, related to the following dates:

- **M1:** Problem Statement, Proof of Concept Plan, Development Plan, Team Charter (Due September 28th)
- **M2:** SRS, Hazard Analysis Revision 0 (Due October 19th)
- **M3:** V&V Plan Revision 0 (Due November 2nd)
- **M4:** Design Document Revision -1 and Fall CD (Due November 16th)
- **M5:** Proof of Concept Demonstration (Completed by December 4th at the latest)
- **M6:** Design Document Revision 0 (Due January 18th)
- **M7:** Revision 0 Demonstration (Completed by February 11th at the latest)
- **M8:** V&V Report Revision 0 and Winter CD (Due March 8th)
- **M9:** Final Demonstration (Completed by March 21st at the latest)
- **M10:** Final Documentation (Due March 31st)
- **M11:** Expo Presentation (Held on April 6th)

Link to our GitHub Project: Link to Kanban Board

10 Proof of Concept Demonstration Plan

The application will be a mobile game in which children learn about paediatric eye procedures by playing through levels. The game will be divided into sections, where each section covers one procedure and is made up of several levels. Quizzes and other short challenges are placed throughout the levels, so the child learns what will happen during the procedure as they play. The full game will also include character customization and a leaderboard to motivate children to keep playing.

The proof of concept (PoC) demonstration will implement one section of the game, covering one common procedure (receiving eye drops), with a small number of playable levels. Each level covers one or more steps of the procedure in the order they happen at the clinic (e.g., sitting in the chair, tilting the head back, the drops going in, blinking, and waiting for the drops to work). Each level includes a short gameplay challenge and ends with a quiz about what the child just learned. The main goal of the PoC is to show that a child can learn about the procedure by playing the game, while also addressing the three highest-priority risks to the project.

The PoC intentionally leaves out character customization, the leaderboard, narration, sound, detailed graphics, advanced animations, and any sections for other paediatric procedures. Keeping the PoC this small lets the team focus on whether the content is correct, whether the levels are fun and teach the procedure, and whether the team can build the game with the chosen technologies, before investing in the remaining game features, detailed art, and audio.

The team expects to create two versions of the game: one for children aged 5–7 and one for children aged 3–5. The PoC targets the 5–7 version first. This version will use simple graphics and short text for the child to read. The 3–5 version, which will need to rely less on reading, will be developed later using what is learned from the PoC.

10.1 Risk 1: Implementing Clinically Accurate Procedure Content

Although the game is meant to be fun, its purpose is to show children what will really happen during their procedure. No team member has clinical training, so the team may not be able to represent these procedures correctly in the levels and quizzes. For example, steps could be in the wrong order, details like eye drops stinging could be left out, the equipment could look different from what is in the clinic, or quiz answers could be wrong. Turning a procedure into gameplay also adds the risk of exaggerating or misrepresenting parts of it to make it more exciting. The content also needs to be easy for young children to understand. If the team gets this wrong, each section will need a lot of rework after clinician review, and the team may run out of time to build all the planned sections.

What the PoC will demonstrate:

- The levels cover every step of receiving eye drops in the order it happens at the clinic.
- The visuals in each level show the equipment and setting the child will actually see at the clinic.
- The gameplay in each level does not misrepresent the procedure (e.g., it does not make the drops seem scary or skip steps to make a level shorter).
- The quiz questions and answers are accurate and only cover what was taught in the level.
- The in-game text is honest about sensations (e.g., drops may feel cold or sting briefly) while staying reassuring, and uses short sentences and familiar words that a child aged 5–7 can read.

Success criteria:

The levels and quizzes are reviewed by a paediatric ophthalmology clinician before the demonstration. The team has the opportunity to show the PoC to a clinician early, so review can begin as soon as the first level is drafted. The clinician confirms that the section accurately reflects what a child experiences. The clinician may have some additional feedback for us, but this will be good enough to revise and present the PoC.

10.2 Risk 2: Child Engagement and Usability

The target users are young children, many of whom cannot read yet, have short attention spans, and are still developing fine motor skills. The game needs to be fun enough that children want to keep playing, while still teaching them about the procedure. If the levels are too difficult, too slow, or not engaging, children will stop playing and will not benefit from the preparation. If the levels are fun but the learning is lost in the gameplay, the game will not meet its purpose. Testing directly with children will likely require research ethics approval, which will not be in place by the PoC date. The team therefore needs a quicker way to check that the game is likely to be fun, usable, and educational for children, without testing with children directly.

What the PoC will demonstrate:

- Each level has a simple gameplay challenge with large touch targets sized for small fingers and controls a young child can learn quickly.
- The child moves through the section level by level, with the next level unlocking after the previous one is completed.
- Each level ends with a short quiz. An incorrect answer gives a gentle hint instead of a failure message, so the child can try again.
- The child can replay any completed level.
- Each level can be completed in about two to three minutes, to fit a young child's attention span.

Success criteria:

During the same review session used for Risk 1, the clinician, who sees young children every day, agrees that a young child would likely enjoy and be able to play the levels (with the planned characters, visuals, and remaining game features added), and identifies any points where a child would lose interest or get confused. The levels are also checked against published guidelines for designing applications for young children (e.g., large touch targets, short sessions, forgiving interactions), with any gaps revised. To show that the game teaches the procedure, a few people unfamiliar with the procedure (e.g., peers outside the team) play through the section and are then able to describe the steps of receiving eye drops and answer most quiz questions correctly. The team ends with a list of concrete changes to make the levels more engaging, and a clear path to testing with children.

10.3 Risk 3: Inexperience with Unfamiliar Technologies

The game is expected to be built in Unity, which the team has limited experience with, and no team member has significant experience with game development or creating game art. The game also needs core systems, such as levels, level progression, and quizzes, that will be reused by every section. There is a risk that learning these technologies takes longer than planned, or that the team cannot produce gameplay and art of sufficient quality in the time available, which would delay every later milestone.

What the PoC will demonstrate:

- Multiple playable levels, with the game moving from one level to the next and saving the child's progress through the section.
- A quiz system that is reused across levels, so that quizzes for future levels and sections can be added without writing new code for each one.
- The art used in the levels (e.g., the clinic setting and equipment) displays correctly in the game.
- The game runs on a physical mobile device.
- The PoC code passes the team's CI pipeline (formatting check and unit tests).

Success criteria:

The levels run smoothly on a physical mobile device, the art displays correctly, progress is saved between levels, and the quiz system works in every level. The team can also estimate how long it takes to build a level, including its art. This estimate will be used to confirm that the planned number of sections and levels, along with character customization and the leaderboard, is realistic for the final game.

10.4 Consequences and Mitigation of an Unsuccessful PoC

If the PoC does not sufficiently address these risks, the consequences for the project and a potential mitigation strategy for each are as follows.

If implementing accurate procedure content (Risk 1) is not resolved:

- **Consequence:** Each section would need several rounds of rework after clinician review, slowing development and reducing the number of procedures the team can cover in the game.
- **Mitigation:** Seek additional clinical reviewers, base the levels and quizzes more closely on existing patient education material from paediatric eye clinics, and keep the gameplay closer to the real steps of each procedure.

If child engagement and usability (Risk 2) are not resolved:

- **Consequence:** The gameplay and level structure would need to be rethought before more sections are built. If players do not learn about the procedure, the game may be entertaining but fail to prepare

children for their appointment.

- **Mitigation:** Simplify and shorten the levels based on the feedback received, tie the gameplay and quizzes more directly to the steps of the procedure, and gather feedback again before building more sections.

If inexperience with unfamiliar technologies (Risk 3) is not resolved:

- **Consequence:** Development would take longer than planned, putting later milestones at risk. The game may have fewer sections and levels, and may not reach the intended visual quality.
- **Mitigation:** Simplify the art style and the gameplay in each level, and rely more on ready-made assets instead of creating custom art.

Whether or not the PoC succeeds, its results will be used to set the number of sections and levels in the game, the gameplay and quiz design, and the art and media approach.

11 Expected Technology

We note at the beginning of this section that the expected technologies listed here can possibly change as the project is developed. That being said, our initial thoughts on implementation details are as follows:

- **Version Control:** Git will be used for version control. The team is already familiar with it, and we all have it setup on our machines.
- **Source Code Hosting:** We will host our source code, as well as other assets and documentation, on GitHub. Again, the team is already familiar with this platform. Additionally, we will be managing our project scheduling using a Kanban board created with GitHub Projects (as detailed in Section 8).
- **Game Engine:** We will use Unity as the engine to develop our game. Motivating choices behind this include the ease at which we can create a cross-platform mobile application (Unity makes it straightforward to deploy on Android and iOS), and that development is done in C#, which some of our team members have good familiarity with. While the team has limited experience with using Unity, there is a robust enough community and documentation that should flatten the learning curve.
- **Programming Language:** We will primarily be using C#. As mentioned above, Unity development is done in C#, thus this is a requirement for using it.
- **Assets:** We will likely make use of Unity's extensive Asset Store, for assets such as sprites, audio, etc.. This is because we are not graphic designers or composers (for example), and it would be less effort intensive to re-use existing assets where we are able. In the cases where we have need for an asset that's not readily available, we will use other tools detailed below to create them ourselves.
- **Blender:** We are tentatively targeting a 3D mobile game, for better alignment with our stretch goal of porting the game to an AR/XR format. For creation of 3D assets not available on the Asset Store, we would make them ourselves in Blender. The motivation behind this selection is that Blender is free and open-source, which keeps our cost and licensing requirements for the project low.
- **Development Environment:** We will use JetBrains Rider as our IDE for Unity development. While normally paid, as students we have the option to use it for free. The main motivation for using Rider is that it has awareness of Unity-specific methods and properties, which makes it easier to debug during development. As well, since the team works cross-platform (some of us have a Mac while others have Windows), Visual Studio is not an option as it is no longer available on Mac. Visual Studio Code also requires extension set-up for C# and Unity development, whereas Rider supports it out-of-the-box, which makes it more appealing.
- **Unit Testing:** Unit testing for game code will be done using the C# NUnit framework for game logic, and rely on the Unity test tools and framework for actual Unity object interactions.
- **CI/CD:** Our CI/CD will be done through GitHub Actions, and consist of two main parts for game

code. First, we will have a "linter gate" stage, that will fail the pipeline if the incoming code fails to meet the code quality standards we lay out as a team. To keep it simple, this will be carried out by the default `dotnet format` command that ships natively with .NET. Second, we will have our unit test stage, which will fail the pipeline if any of the unit test asserts fail. We will use the open source toolkit GameCI to allow us to spin up a headless instance of Unity to test our interaction-focused unit tests.

12 Use of AI and LLMs

For game development, our team plans to use LLMs primarily for assistance with Unity-specific code generation and understanding (when documentation and community tutorials are not sufficient), since we are not very familiar with the game engine. As well, if we need to design any of our own assets (as discussed in Section 10), we will use LLMs to help us understand the platform we are using for asset creation (e.g., Blender) and troubleshoot issues when resources available on the internet are not enough. For code, we will aim to verify the output of our LLMs by referencing existing Unity documentation to confirm functionality is not hallucinated, and seems to be correct given the scenario. We will leverage elements like our linter and unit tests as well. to verify the code conforms to our level of quality and new code does not break existing functionality.

For documentation, our team's primary usage of LLMs will be to carry out LaTeX formatting (when needed), and edit our work clarity and conciseness. We will verify by output of any formatting work by ensuring the documentation compiles correctly, visually inspect it to make sure that it is what we were expecting, and check to see that it does not mess up any other aspects of our document. To verify edits to our written work, we will re-read any edited passages to ensure the meaning of our documentation, as well as crucial information we are trying to convey, is not removed or obscured by the wording changes.

The main tools our team will be using vary between the team members, but we will primarily be using Claude (Claude Code) and ChatGPT (Codex).

13 Coding Standard

The team will follow a consistent set of coding standards to improve readability, maintainability, and consistency across the project. These standards will apply primarily to the C# and Unity codebase and will be reinforced through code review and automated CI checks.

13.1 Naming Conventions

The team will use consistent naming conventions for C# code. Classes, methods, and public properties will use PascalCase, while local variables and private fields will use camelCase. Private fields may use an underscore prefix when appropriate.

Element	**Convention**	**Example**
Class	PascalCase	`EyeDropActivity`
Method	PascalCase	`StartActivity()`
Public Property	PascalCase	`ActivityDuration`
Private Field	camelCase	`activityManager`
Local Variable	camelCase	`attemptCount`
Constant	PascalCase	`MaxAttempts`

Table 7: C# Naming Conventions

13.2 Code Organization

Code will be organized according to functionality so that related components can be easily located and maintained. Classes should have a focused responsibility, and unnecessary duplication should be avoided. Files will be named according to their primary class or component.

Methods should remain reasonably focused and should not contain unrelated functionality. When functionality is reused across multiple components, it should be extracted into an appropriate shared component rather than duplicated.

13.3 Comments and Documentation

Comments will be used to explain non-obvious implementation decisions, assumptions, or reasoning rather than restating what the code already expresses. Public or reusable components should include appropriate documentation where additional context is necessary.

For example, comments should explain why a particular implementation is required rather than simply describing individual statements.

13.4 Error Handling

The application will handle expected errors explicitly and provide meaningful error information when failures occur. Exceptions will not be used for normal program flow, and errors should not be silently ignored. Input and external dependencies will be validated where appropriate to prevent unexpected behaviour.

13.5 Unity Standards

Unity specific code will follow consistent practices across the project. Unity lifecycle methods such as `Awake()`, `Start()`, and `Update()` will be organized consistently within scripts.

Unnecessary work inside frame based methods such as `Update()` will be avoided where possible. Configurable values will use Unity's serialized fields when appropriate rather than unnecessarily exposing fields as public. UI logic, game logic, and data management will be separated where practical to improve maintainability and testing.

13.6 Enforcement

Coding standards will be enforced through a combination of automated tools and peer review. `dotnet format` will be used within the CI pipeline to identify formatting and code style issues. Automated tests and build verification will also be executed as part of CI.

During Pull Request review, team members will verify that changes follow the established naming, organization, documentation, and Unity specific standards. Required CI checks must pass before a Pull Request can be merged. If recurring issues are identified during development, the team may update the coding standards to address them consistently.

14 Appendix — Generative AI Use Disclosure

14.1 AI Tools Used

- ChatGPT (GPT-5.6 Luna)
- Claude (Opus 5.5 Medium)

14.2 How and Where Used

- In Sections 1–4 and Section 16 (Team Charter), the ideas and content were developed by the team. GPT-5.6 Luna was used to improve wording, grammar, and flow while preserving the team's original meaning and decisions.
- In section 10, the PoC plan, risks, and consequences were all identified by the team. Opus 5.5 Medium was used to fix spelling errors and improve grammar.
- In sections 9, 11, and 12, GPT-5.6 Luna was used to ensure LaTeX formatting was correct. Also in section 11, GPT-5.6 Luna was used to brainstorm possible ideas for the development environment, unit testing, and CI/CD technologies to support our Unity workflow.
- GPT-5.6 Luna was used to assist with the LaTeX formatting and presentation of content in Sections 8 and 13.1. This included generating and formatting tables, figures, and code listings based on the team's existing processes, decisions, and coding standards. Specifically:
  - Section 8.1.2: Figure 1
  - Section 8.1.3: Table 3 and Listing 1
  - Section 8.2.2: Table 4
  - Section 8.2.4: Table 5
  - Section 8.2.5: Table 6
  - Section 8.2.6: Listings 2 and 3
  - Section 8.3: Figure 2
  - Section 13.1: Table 7
- GPT-5.6 Luna was also used to review the document for LaTeX formatting issues, inconsistencies, and errors in references, labels, and document structure.

14.3 Verification of AI-Generated Content

All AI-assisted text was reviewed by the team before being included in the document. The team checked that the revised wording accurately represented the team's decisions, policies, and intended meaning. Any changes that did not reflect the team's decisions were removed or revised. Any ideas proposed during brainstorming were independently verified using non-generative AI sources to assess their accuracy and feasibility.

For AI-assisted LaTeX formatting and generated tables, figures, and listings, the team verified that the content accurately reflected the team's actual development workflow, coding standards, and project practices. The team also verified that the document compiled successfully and that the formatting, references, labels, and overall presentation were correct.

The team is responsible for the accuracy, completeness, consistency, and appropriateness of the final document, regardless of whether content was generated or modified with AI assistance.

15 Appendix — Reflection

Mohammad Bilal

Why is it important to create a development plan prior to starting the project?

Creating a development plan gives the team a clear idea of how the project will be organized before development starts. It helps establish expectations for how the team will communicate, divide work, manage the repository, and handle deadlines. Without a plan, some of these decisions would have to be made as problems come up, which could lead to confusion or inconsistent approaches between team members. The development plan also gives the team something to refer back to throughout the project and helps make sure everyone has a shared understanding of how the project should be managed.

In your opinion, what are the advantages and disadvantages of using CI/CD?

One of the main advantages of CI/CD is that it provides an automated way to check new changes before they are merged into the project. Running tests automatically can catch bugs early and help make sure that a new change does not break functionality that was already working. This is especially useful when multiple people are working on the same codebase, since changes from different team members can be tested consistently. CI/CD can also save time in the long run by reducing the amount of manual testing needed for common checks.

The main disadvantage is the additional setup and maintenance required. The team needs to spend time creating the pipeline and making sure it continues to work as the project changes. Failures in the pipeline can also take time to investigate, especially when the issue is with the CI/CD configuration rather than the application itself.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

There were no major disagreements during this deliverable. The main difference in opinion was how GitHub Issues should be structured. One preference was to use higher-level issues, while another was to break them down into more detailed tasks. The team discussed both approaches, agreed on a middle ground, and used a vote to finalize the decision.

Jake Finlay

Why is it important to create a development plan prior to starting the project?

Without a development plan, collaboration can quickly become messy. Each team member tends to bring their own habits for code formatting, pull requests, and commit messages, and nothing ends up being consistent. A standardized workflow makes it much easier to find information in the repository and to contribute in a meaningful way. The plan also acts as a single source of truth for everything related to development. When disagreements come up, the team can point back to what was already agreed upon instead of debating it again. This keeps everyone aligned for the duration of the project.

In your opinion, what are the advantages and disadvantages of using CI/CD?

My favourite advantage of CI/CD is automation. Simply pushing to a branch can trigger PDF generation, builds, deployments, tests, formatting, and linting without any manual effort. The results of these checks are visible to everyone, so reviewers can approve pull requests with much more confidence. However, CI/CD also has some disadvantages. A poorly configured pipeline can take a long time to run, which becomes frustrating when checks must pass before a merge is allowed. Pipelines can also break, and a broken check may let quality issues slip through or block progress entirely. As a result, someone has to be responsible for maintaining the pipeline. That maintenance draws on team resources that could otherwise go toward development.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our group did not have any real disagreements during this deliverable. There was some back-and-forth about the contents of the development plan in our team group chat, but these discussions were resolved quickly. Going forward, I expect that clear communication and good documentation will help us settle any development-related issues that arise.

Jackson Beach

Why is it important to create a development plan prior to starting the project?

I believe a development plan is important as it lays out key aspects of the project at various levels. At a high level, we lay out guidelines for team meetings, communication, roles, and project milestones, so we are clear on where we are headed and how we will accomplish it together as a team. At a lower level, we lay out our software development workflow, standards, and (preliminary) technologies we will use so we are all clear on (what we consider) the best and most efficient way to implement our project is. Without one, we would all likely do our own thing in terms of contributions, which would be confusing and probably lead to conflict among members. Having something like this laid out is meant to reduce conflict by providing a "ground truth" we can all reference.

In your opinion, what are the advantages and disadvantages of using CI/CD?

The main advantage I see for using CI/CD is that it can help prevent breaking changes from being merged in, which alleviates the effort needed to fix those bugs in the production build later. Automated linting and regression testing saves the developer from having to manually check for issues, which can be time consuming and susceptible to human error. The main disadvantage I see is that it can be a pain to configure and maintain, as it requires careful configuration of your YAML files and required credentials/secrets. Also, debugging errors with things like runners can be confusing, as it can be unclear exactly where the problem is coming from.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

I wouldn't say there were any disagreements. Most discussion points revolved around what we felt was the most efficient and ideal development workflow for the team. We had conversations on what would be the most clear for commit messages, how to structure our pull requests, etc., but we were able to come to a consensus on these items. We are all coming from a place of ensuring that what we do during development is simple and straightforward so we can spend more of our time and energy on actually developing the game.

Ahsan Muzammil

Why is it important to create a development plan prior to starting the project?

Creating a development plan before starting the project is important because it establishes a shared understanding of how the team will organize, track, and integrate its work. Without a defined workflow, different team members may make different assumptions about how tasks should be completed, reviewed, and merged. Developing the plan also allows the team to identify potential issues with collaboration before they occur. For example, defining our branching strategy, issue management process, Pull Request requirements, and CI/CD process in advance provides a consistent structure for development and makes responsibilities clearer. I also learned that a development plan is not only about deciding which tools to use, but also about establishing how those tools work together to support the team's overall development process.

In your opinion, what are the advantages and disadvantages of using CI/CD?

In my opinion, one of the main advantages of CI/CD is that it provides automated verification of changes before they are integrated into the shared codebase. This is particularly valuable based on my experience

during my previous co-op term. The company I worked for did not have an effective CI/CD pipeline, so whenever I received feedback from a user on a feature request and made changes to address that feedback, I had to manually rerun the relevant tests after every change. This process could take a significant amount of time, especially when multiple rounds of feedback were required. Much of this time could have been reduced if the tests had been automatically executed through a CI pipeline after each change. This experience showed me how the lack of CI/CD can result in repetitive manual work that does not directly contribute to implementing the requested changes.

For our project, automated formatting checks, unit tests, and build verification can identify problems earlier and reduce the amount of manual verification required from individual team members. CI/CD also provides a consistent process for validating changes, which is particularly useful when multiple team members are working on different parts of the project.

However, CI/CD also introduces additional setup and maintenance overhead. The team needs to spend time creating and maintaining the workflows, configuring the required tools, and addressing failures when the CI environment behaves differently from a local development environment. There can also be an initial learning curve when setting up tools such as GitHub Actions and GameCI. Despite these disadvantages, my previous co-op experience has shown me that automating repetitive verification can save significant development time, particularly when changes need to be made and tested repeatedly.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our main disagreement during this deliverable was related to our overall Git workflow. We initially had different understandings of how our branching strategy was supposed to work. Some group members assumed that we would use a traditional workflow where each team member creates a branch for their assigned task, creates a Pull Request to an epic branch, and eventually merges the completed epic branch into the main branch. Other members understood that we would use a fork-based workflow, where each team member works within their own fork and submits Pull Requests to the primary repository.

We resolved this disagreement by discussing both approaches as a group rather than assuming that one interpretation was correct. Each approach was considered based on how well it would support parallel development, code review, repository protection, and collaboration. After discussing the advantages and disadvantages of both approaches, we held a group vote. The vote was unanimous in favour of using the fork-based workflow.

This disagreement showed me the importance of explicitly discussing development processes before implementation begins. Even though everyone understood the general purpose of branches and Pull Requests, we had different interpretations of how they should be combined into an overall workflow. Establishing and documenting the workflow early helped ensure that everyone had the same expectations before continuing with development.

Ethan McMehen

Why is it important to create a development plan prior to starting the project?

I think it is important to create a development plan prior to beginning the project for three reasons: everyone is aligned on documentation standards, teamwork expectations are set out clearly, and development technologies are all agreed upon to ensure effective communication. A challenge I found we faced as a team that helped teach us the important of this documentation is our GitHub workflow. We had multiple discussions on how we wanted to record issues and use them because from our co-ops we all had different experiences. Ultimately we all became aligned and we each have our own fork of the repo. From this experience, I learned that we saved ourselves a lot of stress and disagreements in the future due to this communication and plan. Had we begun the project without convening this information, everyone would have had different process, standards, and review processes increasing the complexity of our work. This would have complicated everything and added many unproductive meetings and discussions, which could foster negative emotions within the team. But developing this document made me feel comfortable with my team mentality and standards

we have all agreed to.

In your opinion, what are the advantages and disadvantages of using CI/CD?

Being completely honest, I had never heard of CI/CD before Capstone because I did not use in my co-op and in previous courses it was not used. Due to this, I am learning CI/CD completely from scratch, which is good because I can see the technical advantages and the disadvantages the tool can develop within workflow. The first huge advantage I see is the ability to integrate testing through CI into the pipeline to provide an extra verification layer on coding bugs. If the PR requested fails the CI tests, it shows something not caught be developers or testers and this poses a huge development in the coding sense of the software development process. On the other hand, this in tight deadline's and quick agile methodologies could yield to a lower standard on testing because CI could/would be used as the main testing core. Essentially, the disadvantage is lazy test engineering which could be supported through the dependency of CI. Another advantage I see though is through our current implementation by the TAs of generating PDFs, simplifying the LaTeX process for us saving time. But the disadvantage is the learning curve as this is a functionality I don't know how to implement. Overall, I see the advantages in increasing productivity but to prevent it becoming a dependency and thus a disadvantage I will have to properly learn how it functions.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our team did not have any disagreements but more different perspectives. As mentioned before, we all had different ideas on how to label issues and produce branches. Some of us through that a structure of Parent issue to subsection to subsubsection was best, while others thought deliverable to section to subsection would be best. After discussion, we decided to follow a combination of both where the deliverable would be made into the epic branch, and we would do issues from section to subsection. This scheme made the hierarchy more relaxed while ensuring structure. Ultimately, I found our ability to discuss as a team and draw a few depictions of our thoughts was extremely positive. Everyone was open to finding a solution for everyone and no one was authoritarian on their opinion. In this way, I learned more about how I can trust my teammates as long as good communication is provided.

16 Appendix — Team Charter

External Goals

The team's main goal is to develop a project that is useful, well-designed, and worth presenting at the end of the year. Since the project addresses a real clinical problem, the final product should reflect the needs of its users and stakeholders rather than focusing only on the technical implementation. The project will also provide the team with experience working with clinicians and developing software for a real-world setting. A secondary goal is to achieve a strong academic result in the course.

Attendance

Expectations

Team members are expected to attend scheduled meetings on time and stay for the full meeting unless they have communicated otherwise. Members should come prepared by reviewing any material relevant to the meeting and completing work that was assigned beforehand. If someone cannot attend, they should notify the team as early as possible and review the meeting notes afterward. Missing meetings occasionally is understandable, but missing three consecutive meetings without a valid reason will result in the issue being discussed with the team.

Acceptable Excuse

Acceptable reasons for missing a meeting or deadline include illness, medical appointments, family emergencies, academic obligations that cannot be rescheduled, and other significant personal circumstances. Unexpected work-related commitments may also be considered when they cannot reasonably be avoided. Whenever possible, the team member should provide notice before the conflict occurs.

Missing a meeting or deadline due to a lack of preparation, forgetting, or a conflict that could have been avoided will not be considered an acceptable excuse. A member who misses a meeting or deadline is responsible for reviewing what was missed and completing any outstanding work.

In Case of Emergency

If an emergency prevents a member from attending a meeting or completing assigned work, they should notify the team through the team Discord chat as soon as they are able. If the member is unable to communicate, notification should be provided when they are able to do so. The team will determine whether the affected work can be redistributed temporarily or whether deadlines need to be adjusted. If the situation continues for more than two weeks and affects the team's ability to meet project deadlines, the teaching assistants and course instructors will be contacted to determine an appropriate approach.

Accountability and Teamwork

Quality

Team members are expected to take responsibility for their assigned work and ask for help when needed. Assigned tasks should be completed before the relevant team meeting so that progress can be reviewed and discussed. Every contribution to the repository will be reviewed by at least one other team member before it is merged. Reviews should provide meaningful feedback and ensure that the work meets the project requirements and agreed-upon standards. Major deliverables will be reviewed by the team against the course rubric before submission, with the goal of meeting the Level 4 criteria.

The team's internal deadline for deliverables will be two days before the official submission deadline. This will provide sufficient time to review the deliverable, address feedback, and make any necessary revisions before submission.

As the project involves clinical stakeholders, particular care will be taken when handling information or feedback obtained from them. Any concerns regarding privacy, stakeholder feedback, project requirements,

or decisions that may affect the users or functionality of the application should be communicated to the team and addressed before proceeding.

Attitude

Team members are expected to maintain a respectful, positive, and collaborative attitude throughout the project. Everyone should contribute their ideas, listen to the opinions of others, and be open to constructive feedback. Disagreements are expected to be handled respectfully and focused on the work rather than the individual. Team members should also communicate openly when they have concerns or need assistance so that issues can be addressed without affecting the team dynamic.

Stay on Track

The team will stay on track through regular check-ins during team meetings to discuss completed work, current tasks, and any issues that may affect upcoming deadlines. GitHub Issues and Projects will be used to break down larger milestones into smaller tasks, assign responsibilities, and track progress.

The team will aim to complete deliverables two days before the official deadline. This will allow time to review the work, compare it against the course rubric, address concerns, and make any necessary revisions.

Members are expected to complete their assigned tasks within the agreed-upon timeframe and communicate any delays as early as possible. If a member's contribution is consistently below expectations, the issue will first be discussed with the member. If the situation does not improve, the team may seek advice from the teaching assistant and, if necessary, the course instructor. Failure to contribute a fair share of the work may also be reflected in peer evaluations.

Target Metrics:

- Attendance: 90% of scheduled team meetings.
- Issues: All members must contribute to issues for each deliverable, with their contributions reflecting approximately 20% of the team's total workload.
- Code review: Every repository contribution must be reviewed by at least one other team member before being merged.

If a member does not meet a target, they should communicate the reason to the team and identify how the issue will be addressed. If targets are repeatedly not met, the team will meet with the teaching assistant to discuss how to improve participation and maintain an equitable distribution of work.

Team Building

The team will have a monthly social activity to maintain team cohesion throughout the project. The activity will be planned based on the availability and preferences of the team members.

Decision Making

The team will make decisions through discussion and majority vote. Each member will have the opportunity to share their opinion before a vote is held. If a disagreement cannot be resolved within the team, a neutral mediator, either a team member or someone outside the team such as a teaching assistant, will be consulted.